SCCAD 3nm PDK

User Manual

Physical Design with Synopsys IC Compiler 2

Version	v1.0
Release Date	TBD
Organization	SCCAD Lab, University of Southern California
Tool	Synopsys IC Compiler 2 (ICC2)
Contact	sw.jung@gatech.edu (Sungwoo Jung)

This manual provides comprehensive guidance for users working with the **SCCAD 3nm Process Design Kit (PDK)**. It covers PDK installation from GitHub, environment configuration, and step-by-step instructions for running Place-and-Route (PnR) using **Synopsys IC Compiler 2 (ICC2)**.

1 Introduction

1.1 Overview of SCCAD 3nm PDK

The SCCAD 3nm Process Design Kit (PDK) is developed and maintained by the SCCAD Laboratory to support academic and research-oriented digital IC design flows at the 3 nm technology node. The PDK provides a complete set of design enablement files — including technology files, standard-cell libraries, SPICE models, and interconnect models — that allow designers to perform synthesis, place-and-route (PnR), and sign-off verification using industry-standard EDA tools.

Several open-source 3 nm PDKs are available to the research community. The table below positions SCCAD-3nm alongside FreePDK3 (NCSU) and GT3 (Gatech) to highlight the distinguishing capabilities of this release.

Feature	FreePDK3 (NCSU)	GT3 (Gatech)	SCCAD-3nm
Tech Node	3nm	3nm	**3nm**
Device Type	GAAFET	GAAFET	**GAAFET**
Cell Heights	5.5T	6T	**6T, 5T**
Power Rail (PR)	Front-side (FSPR)	Buried (BPR)	**FSPR, BPR**
Support Back-side Metal	No	No	**Yes**

As shown in the table, all three PDKs target the 3 nm node and adopt Gate-All-Around FET (GAAFET) device architectures. Within this shared foundation, the SCCAD-3nm PDK offers several capabilities that extend the design space available to researchers.

- **Dual cell-height support (6T and 5T).** Providing both 6-track and 5-track standard-cell libraries gives designers flexibility to trade off area density against routability.
- **Both Front-side and Buried Power Rail (FSPR & BPR).** Supporting both power delivery architectures allows direct comparison of conventional front-side routing against buried power rail approaches.
- **Back-side metal support.** SCCAD-3nm includes back-side metal interconnect, enabling exploration of back-side power delivery network (BS-PDN) and signal routing techniques.

1.2 Key Features and Technology Highlights

Technology Node: 3 nm GAAFET

The SCCAD-3nm PDK is built on a **3 nm Gate-All-Around FET (GAAFET)** process. Unlike FinFET, where the gate wraps around three sides of a fin, GAAFET surrounds the channel on all four sides using stacked nanosheet or nanowire structures. This architecture provides superior electrostatic control, reduced short-channel effects, and improved drive current at scaled supply voltages. The fabrication process sequence is illustrated in Figure 1.

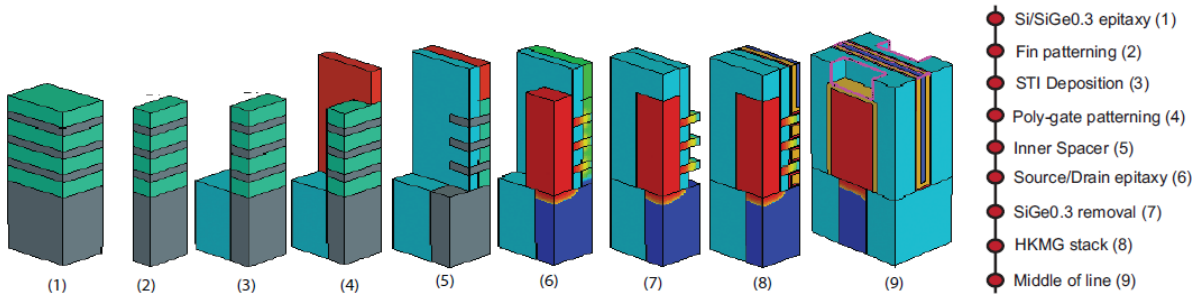


Table 1 summarizes the key device physical parameters of the SCCAD-3nm NSFET device in comparison with a representative 5 nm FinFET process.

Table 1. Comparison of device physical parameters of FinFET and NSFET device.

Physical Parameters (nm)	TSMC 5nm	NSFET 3nm
CPP	48	**42**
Fin/NS pitch	25	**NA**
Gate length	15	**12**
Spacer length	6	**5**
# Fin/NS	2	**3**
Fin/NS thickness	6	**5**
Fin/NS width	NA	**30, 20**
Fin/NS stack height	55	**55**

Standard-Cell Library

The SCCAD-3nm PDK includes two standard-cell libraries. The **6-Track (6T) library** uses a Front-side Power Rail (FSPR) architecture, and the **5-Track (5T) library** adopts a Buried Power Rail (BPR) architecture. The complete cell set comprises **57 standard cells**. Table 2 lists the full cell set along with drive strength variants for each cell type.

Table 2. Standard-cell library overview.

STD Cells	(Input) x (# parallel Trs)
INV	x1, x2, x3, x4, x5, x6, x7, x8, x10, x12, x14, x16
BUF	x1, x2, x3, x4, x5, x6, x7, x8, x10
NAND, NOR	2x1, 3x1, 4x1, 2x2, 3x2
AND, OR	2x1, 3x1, 4x1
XOR, XNOR	2x1, 3x1
AOI, OAI	(21, 22, 221, 222, 311, 31) x1
MUX	2x1
D Flip-Flop	Hx1 (async.), HQNx1 (sync.)
DHL (latch)	x1
Total	**57**

Each cell type in the library serves a distinct role in digital circuit design:

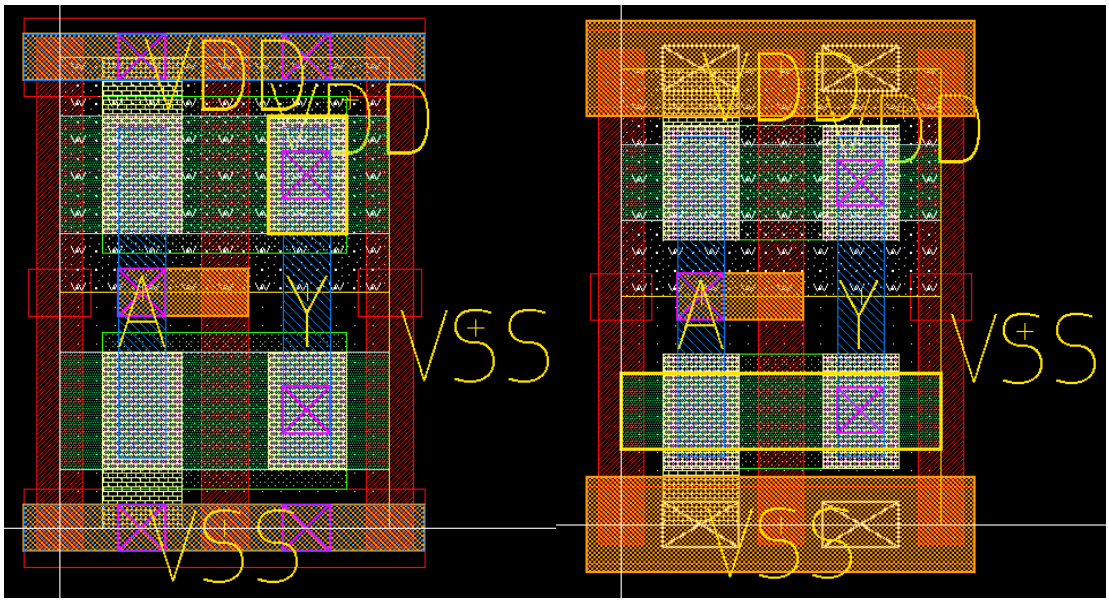
- **INV / BUF** — Inverter and buffer cells provided in 12 and 9 drive strength variants, forming the backbone of clock tree synthesis and signal-integrity-driven buffering.
- **NAND / NOR** — 2-, 3-, and 4-input gates supporting both area-optimized and performance-optimized implementations.
- **AND / OR** — Non-inverting gates in 2-, 3-, and 4-input configurations.
- **XOR / XNOR** — Essential for arithmetic datapaths, parity generation, and error-detection logic.

- **AOI / OAI** — Complex gates in six configurations enabling compact, high-speed multi-level boolean function implementation.
- **MUX** — A 2-to-1 multiplexer for datapath and control-flow selection.
- **D Flip-Flop** — Asynchronous reset type (Hx1) and synchronous reset type with complementary output (HQNx1).
- **DHL (Latch)** — A level-sensitive D-latch for time-borrowing and latch-based pipeline designs.

Figure 2 shows the layout of the **INVx1** cell for each library variant.

6-Track (6T) / Front-Side Power Rail (FSPR): In the 6T library, the power rails (VDD and VSS) are routed on the M1 metal layer, each rail being 3x the Critical Dimension (CD) wide. The resulting cell height is 144 nm. The nanosheet width is fixed at 30 nm.

5-Track (5T) / Buried Power Rail (BPR): In the 5T library, the power rails are moved to the buried metal layer MBPR, connected through VBPR vias (12 nm x 18 nm). The MBPR metal width is 25 nm. The effective cell height is reduced to 120 nm.



[FIGURE 2a/b: INVx1 layout of 6T FSPR and 5T BPR library]

Back-end-of-Line (BEOL) Technology

The SCCAD-3nm PDK features a comprehensive BEOL stack from **MB2 through M9** and via layers **VB1 through V8**. Table 3 summarizes the key physical parameters.

Table 3. BEOL Metal and Via Layers — Physical Parameters.

Tier	Metal	W (nm)	Resistance (Ω)
Back-side	MB2-MB1	61	11
BPR cell layer	MBPR	25	65
Front-Side	M0	20	523
	M1-M3	12	347
	M4, M5	18	101
	M6, M7	24	44

Tier	VIA	W x L (nm²)	Resistance (Ω)
Back-Side	VB1	40 x 40	4.0
BPR cell layer	VBPR	20 x 12	56
BSP cell layer	µ-TSV	60 x 60	5
	BSC	20 x 20	74.9
Front-Side	V0-V3	12 x 12	63.5
	V4, V5	18 x 18	19.38
	V6, V7	24 x 24	10.8

Among the layers listed above, the SCCAD-3nm PDK provides dedicated back-side metal interconnect through two layers: **MB1** and **MB2**. These layers are available for both Power Delivery Network (PDN) routing and general signal routing.

Using MB1 and MB2 for PDN distribution — commonly referred to as a **Back-Side Power Delivery Network (BS-PDN)** — decouples power routing from signal routing, reducing IR drop and freeing front-side metal resources for timing-critical signals.

EDA Tool Support

The SCCAD-3nm PDK is validated with the following industry-standard EDA tools. Two separate user manuals are provided — one for the Synopsys ICC2 flow (this document) and one for the Cadence Innovus flow.

2 Getting Started — PDK Installation from GitHub

2.1 Cloning the Repository

The SCCAD 3nm PDK is hosted on GitHub. Use the following commands to clone the repository to your local workstation or server.

HTTPS:

```
git clone https://github.com/sjung366/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

SSH (recommended for development):

```
git clone git@github.com:SCCAD-Lab/SCCAD-3nm-PDK.git
cd SCCAD-3nm-PDK
```

If the repository uses Git submodules, initialize them as follows:

```
git submodule update --init --recursive
```

2.2 Repository Structure Walkthrough

The repository is organized into three top-level directories: PDK-Synopsys, PDK-Cadence, and Sample Designs. This manual covers the PDK-Synopsys flow. Each version (Front-side / Back-side) has an identical folder structure.

```
SCCAD-3nm-PDK/
├── PDK-Synopsys/
│   ├── Front-side Version/
```

```
│   │   ├── PDK Development/
│   │   │   ├── DRC/            # DRC rule decks (IC Validator)
│   │   │   ├── LVS_PEX/        # LVS/PEX decks and StarRC config
│   │   │   ├── MODEL/          # HSPICE SPICE model cards
│   │   │   ├── SCRIPTS/        # PDK build and utility scripts
│   │   │   └── STD-CELLS/      # Standard cell GDS layouts
│   │   └── PnR/
│   │       ├── TF/             # Technology file for ICC2
│   │       ├── TECH-LEF/       # Technology LEF
│   │       ├── LEF/            # Cell LEF
│   │       ├── LIB_DB/         # Liberty (.lib) and DB (.db) files
│   │       ├── NDM/            # Pre-built ICC2 NDM reference library
│   │       └── ITF/            # ITF + TLUPlus + nxtgrd RC tables
│   └── Back-side Version/      # Identical structure (BPR variant)
├── PDK-Cadence/
│   ├── Front-side Version/     # Identical structure, QRC instead of ITF
│   └── Back-side Version/
└── Sample Designs/
    ├── ecg/
    │   ├── ecg.netlist.v
    │   ├── ecg.sdc
    │   └── ecg_final_design.def
    └── openpiton/
        ├── HARD-LEF (OpenPiton)/
        ├── tile.netlist.v
        ├── tile.sdc
        ├── openpiton_final_design.def
        └── Layouts of sample designs.png
```

The table below describes the role of each key directory under PDK-Synopsys/Front-side Version/PnR/:

Directory	Description
PnR/TF/	Technology file (.tf) defining routing layers, via rules, and design rule constraints for ICC2
PnR/TECH-LEF/	Technology LEF file defining layer names, routing directions, and spacing rules
PnR/LEF/	Cell LEF files providing macro abstracts (pin locations, obstructions, cell dimensions)
PnR/LIB_DB/	Liberty (.lib) files with timing arcs and power tables; compiled .db files for DC and PT
PnR/NDM/	Pre-built ICC2 New Data Model (NDM) reference library
PnR/ITF/	ITF for StarRC parasitic extraction; TLUPlus and nxtgrd RC tables
PDK Development/MODEL/	HSPICE SPICE model cards for NMOS/PMOS across all PVT corners
PDK Development/DRC/	DRC rule decks for Synopsys IC Validator
PDK Development/LVS_PEX/	LVS decks and StarRC parasitic extraction configuration
Sample Designs/	ECG and OpenPiton benchmark designs with netlist, SDC, and final DEF

2.3 Environment Setup

Before running any EDA tool, the following environment variables must be configured manually. These variables must be set in your shell environment (e.g., **.bashrc** or **.cshrc**) or exported at the beginning of each session.

Required Environment Variables (Front-side Version):

```
# Root directory of the cloned PDK repository
export PDK_ROOT=/path/to/SCCAD-3nm-PDK

# Synopsys Front-side PnR base path
export PNR_ROOT="$PDK_ROOT/PDK-Synopsys/Front-side Version/PnR"

# Technology file path (ICC2)
export TECH_PATH="$PNR_ROOT/TF"

# Standard-cell NDM library path
export STDCELL_NDM="$PNR_ROOT/NDM"

# Liberty timing library path
export LIB_PATH="$PNR_ROOT/LIB_DB"

# LEF file path
export LEF_PATH="$PNR_ROOT/LEF"

# StarRC ITF path
export ITF_PATH="$PNR_ROOT/ITF"

# SPICE model path
export MODEL_PATH="$PDK_ROOT/PDK-Synopsys/Front-side Version/PDK Development/MODEL"
```

Note: For the Back-side Version (BPR), replace "Front-side Version" with "Back-side Version" in the paths above.

After setting the variables, verify that the key PDK files are accessible:

```
echo $PDK_ROOT
ls "$TECH_PATH"
ls "$STDCELL_NDM"
ls "$LIB_PATH"
ls "$ITF_PATH"
```

Required EDA Tools:

The following EDA tools must be installed and accessible via your system PATH. License servers should be configured according to your institution's setup.

- **Synopsys IC Compiler 2 (ICC2)** — Primary tool for place-and-route. The **icc2_shell** executable must be accessible.
- **Synopsys StarRC** — Parasitic extraction tool. The **StarXtract** executable must be accessible.
- **Synopsys PrimeTime (PT)** — Static timing analysis and sign-off tool. The **pt_shell** executable must be accessible.
- **Synopsys Design Compiler (DC)** — Logic synthesis tool. The **dc_shell** executable must be accessible. (Optional if a pre-synthesized netlist is provided.)

System Requirements:

Item	Requirement
Operating System	RHEL 7/8, CentOS 7, or compatible Linux (64-bit)
Shell	bash or csh/tcsh
TCL	Version 8.5 or later (required by ICC2 and PT)
Python	Version 3.8 or later (required for utility scripts)
Disk Space	Minimum 10 GB free for PDK files and run directories

Memory	Minimum 32 GB RAM recommended for full PnR runs

License Configuration: Synopsys tools require a valid license server. Set the license server address using the **SNPSLMD_LICENSE_FILE** or **LM_LICENSE_FILE** environment variable:

```
export SNPSLMD_LICENSE_FILE=27020@your.license.server

# Verify tool availability
which icc2_shell
which StarXtract
which pt_shell
```

3 Running PnR with Synopsys IC Compiler 2

3.1 Essential PDK Files for PnR

Before launching a PnR run, it is important to understand which PDK files are loaded at each stage of the flow. The table below lists the essential files required for a complete ICC2-based PnR run.

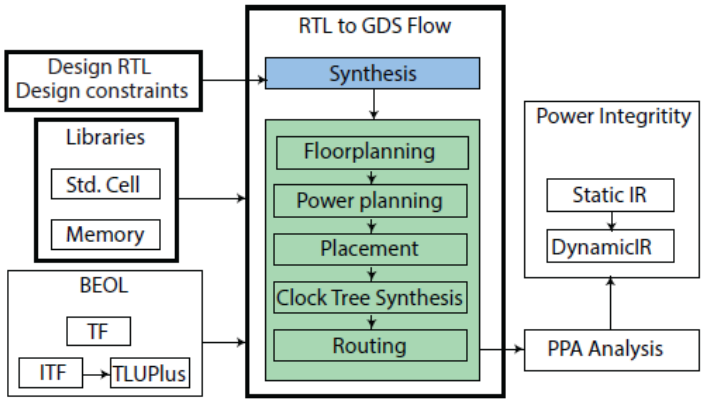
Directory / File	Format	Tool	Description
PDK-Synopsys/Front-side Version/PnR/TF/3nm_GAA_FSPR.tf	.tf	ICC2	Technology rules: routing layers, vias, design rules
PDK-Synopsys/Front-side Version/PnR/TECH-LEF/3nm_GAA_FSPR_tech.lef	.lef	ICC2	Technology LEF: layer definitions and via rules
PDK-Synopsys/Front-side Version/PnR/LEF/3nm_GAA_FSPR.lef	.lef	ICC2	Cell LEF: macro dimensions and pin geometry
PDK-Synopsys/Front-side Version/PnR/LIB_DB/3nm_GAA_FSPR_lvt_nldm.lib	.lib, .db	ICC2 / PT	Liberty: timing arcs, power tables
PDK-Synopsys/Front-side Version/PnR/NDM/3nm_GAA_FSPR.ndm	.ndm	ICC2	Pre-built ICC2 reference library (NDM format)
PDK-Synopsys/Front-side Version/PnR/ITF/*.itf	.itf, .tluplus, .nxtgrd	StarRC	Interconnect Technology File for RC extraction

The technology file (.tf) and technology LEF together define the physical design rules of the process. The **cell LEF** provides the abstract view of each standard cell. Combined with the **Liberty (.lib / .db)** files, these allow ICC2 to perform timing-driven placement and optimization.

The NDM (New Data Model) library is a pre-compiled ICC2-native format that packages the technology file, LEF, and Liberty data into a single reference library. The **ITF and associated RC files (.tluplus, .nxtgrd)** are consumed by Synopsys StarRC during parasitic extraction.

3.2 PnR Flow Overview

The SCCAD-3nm PDK supports a complete RTL-to-GDS back-end flow using Synopsys IC Compiler 2. The overall flow is illustrated in **Figure 3**, spanning from RTL synthesis through physical implementation and PPA analysis. Each stage is described in detail below.



[Figure3. Overall PnR Flow used in this work]

Step 1 — RTL Synthesis

The flow begins with logic synthesis using **Synopsys Design Compiler (DC)**. The user provides RTL source files (Verilog/SystemVerilog) together with design constraints in SDC format. DC maps the RTL onto the SCCAD-3nm standard-cell library and produces a gate-level netlist and synthesized SDC file, which are then passed to ICC2.

```
dc_shell -f scripts/dc/run_syn.tcl \
         -output_log_file logs/syn.log
```

Step 2 — Design Setup in ICC2

Before physical implementation begins, the design library must be initialized in ICC2 using the **New Data Model (NDM)** format. The pre-built NDM reference library packages the technology file, LEF, and Liberty data into a single file.

```
# Open or create the design NDM library
create_lib -technology "$TECH_PATH/3nm_GAA_FSPR.tf" \
           -ref_libs   "$STDCELL_NDM/3nm_GAA_FSPR.ndm" \
           my_design.ndm

# Read synthesized netlist and constraints
read_verilog  results/syn/netlist.v
read_sdc      results/syn/constraints.sdc
link_design   $TOP_MODULE
```

Step 3 — Floorplanning and Power Planning

Floorplanning defines the die area, core utilization, and the placement of any hard macros or memory blocks. Power planning follows immediately, creating the VDD/VSS power mesh across the design.

```
# Initialize floorplan
initialize_floorplan -utilization 0.70 \
                     -core_offset  {2 2 2 2}

# Create power mesh
create_pg_mesh_pattern -layers {M4 M5} \
                       -nets   {VDD VSS}
compile_pg
```

Step 4 — Placement

ICC2 performs timing-driven placement using **place_opt**, which simultaneously optimizes cell placement, logic restructuring, and buffer insertion to meet timing constraints.

```
place_opt
```

Step 5 — Clock Tree Synthesis (CTS)

Clock Tree Synthesis builds the clock distribution network to minimize skew and meet insertion delay targets. ICC2's **clock_opt** command performs CTS and post-CTS optimization in a single step.

```
# Run CTS and post-CTS optimization
clock_opt
```

Step 6 — Routing

Global and detail routing are performed by **route_auto**, followed by post-route optimization with **route_opt**. The router uses the metal stack and spacing rules defined in the PDK technology file.

```
route_auto
route_opt
```

Step 7 — PPA Analysis

After routing and parasitic extraction, the final Power, Performance, and Area (PPA) metrics are evaluated. **Performance** is assessed through static timing analysis (STA) using **Synopsys PrimeTime (PT)**, reporting worst negative slack (WNS) and total negative slack (TNS). **Power** analysis is performed in PT-PX. **Area** is reported directly from ICC2 after routing.

```
# Static timing analysis and power analysis — PrimeTime
pt_shell -f scripts/pt/run_sta.tcl \
         -output_log_file logs/sta.log

# Area report — ICC2
report_design
report_utilization
```

Upon successful sign-off, the final GDS layout can be streamed out from ICC2 for DRC/LVS verification:

```
write_gds -output results/final/design.gds
```

4 Troubleshooting

This chapter describes common issues encountered when setting up and running the SCCAD-3nm PDK with Synopsys ICC2, along with their recommended solutions.

4.1 Installation and Environment Issues

PDK_ROOT not set or files not found

Symptom: Commands like ls $TECH_PATH return empty results or "No such file or directory".

Solution: Verify that PDK_ROOT is exported correctly and points to the repository root. Re-source your shell configuration file:

```
echo $PDK_ROOT           # Should print the full path
source ~/.bashrc         # Re-load environment variables
ls $PDK_ROOT/TF          # Verify TF directory exists
```

License server error on tool startup

Symptom: ICC2, DC, or PT exits immediately with "license checkout failed" or "cannot connect to license server".

Solution: Check that the license server address is set correctly and the server is reachable:

```
echo $SNPSLMD_LICENSE_FILE     # Should print port@server
ping your.license.server      # Check network connectivity
lmstat -a -c $SNPSLMD_LICENSE_FILE  # Check available licenses
```

4.2 NDM Library Issues

NDM library creation fails

Symptom: create_lib returns an error referencing the technology file or reference library.

Solution: Verify the paths to the technology file and NDM reference library are correct, and that the files exist:

```
ls $TECH_PATH/3nm_GAA_FSPR.tf       # Check TF file exists
ls $STDCELL_NDM/3nm_GAA_FSPR.ndm    # Check NDM file exists

# In icc2_shell, use absolute paths if relative paths fail
create_lib -technology /full/path/to/3nm_GAA_FSPR.tf \
           -ref_libs   /full/path/to/3nm_GAA_FSPR.ndm \
           my_design.ndm
```

Liberty version mismatch warning

Symptom: ICC2 prints warnings about Liberty file version or missing timing arcs after read_lib.

Solution: Ensure you are loading the correct .db file (compiled from .lib using lc_shell). If only the .lib file is available, compile it first:

```
# Compile .lib to .db using Library Compiler
lc_shell
read_lib $LIB_PATH/3nm_GAA_FSPR_lvt_nldm.lib
write_lib sccad3nm_lvt -format db \
          -output $LIB_PATH/3nm_GAA_FSPR_lvt_nldm.db
exit
```

4.3 PnR Flow Issues

Placement congestion / utilization too high

Symptom: place_opt reports high congestion or the design cannot be routed cleanly.

Solution: Reduce core utilization or increase die area. For BPR designs, ensure the power mesh does not block cell placement:

```
# Reduce utilization from 0.70 to 0.65
initialize_floorplan -utilization 0.65 \
                     -core_offset  {2 2 2 2}
```

Timing not closed after route_opt

Symptom: PrimeTime reports negative WNS after routing even after multiple route_opt iterations.

Solution: Check the clock constraints in your SDC file. Common causes are an overly tight clock period or missing clock uncertainty. Also verify that the correct timing library corner (TT/SS/FF) is loaded:

```
# In PrimeTime, check worst path
report_timing -max_paths 10 -path_type full

# Check which library corner is active in ICC2
report_lib
```

StarRC extraction fails or produces empty SPEF

Symptom: StarRC exits with an error or the resulting SPEF file is empty / has no parasitics.

Solution: Verify that the ITF path and the routed DEF file are correctly specified in the StarRC config file:

```
# Check StarRC config file
cat scripts/starrc/starrc.cfg

# Key fields to verify:
# TCAD_GRD_FILE: path to .nxtgrd file
# MAPPING_FILE:  layer mapping from LEF to ITF
# NETLIST_TYPE:  should be SPEF
```